

ChatGPT

what happen when we do this

```
const app = express();  
const server = http.createServer(app);
```

specially in socket io because app is already in http so why we pass in createServer and what happen when we do this ?

Great question 👍 This confuses many people when first using **Socket.IO with Express**.

Let's break it down step-by-step.

1 What happens here?

JavaScript

```
const app = express  
const server = http.      app
```

Step 1: Create Express app

JavaScript

```
const app = express
```

`app` is just a request handler function, not an actual HTTP server.

Internally Express works like this:

JavaScript

```
function app req res  
  // handle request
```

So `app` is basically a function that handles requests.

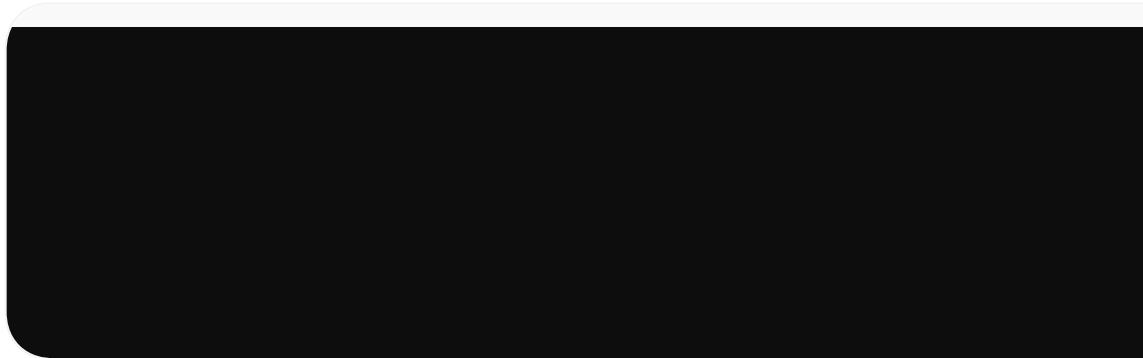
Step 2: Create HTTP server

JavaScript

```
const server = http.createServer(app)
```

Here Node.js creates a **real HTTP server** and attaches Express to it.

Conceptually it becomes:



So when a request comes:



2 Then why not just use `app.listen()` ?

Normally we do this:

```
</> JavaScript
```

```
app.listen(3000)
```

But internally Express actually does this:

```
</> JavaScript
```

```
http.createServer(app).listen(3000)
```

So both are almost the same.

3 Why Socket.IO requires `createServer()`

Socket.IO needs **direct access to the HTTP server** because it upgrades the connection to **WebSocket**.

Example:

```
</> JavaScript
```

```
const express = require("express")
const http = require("http")
```

```
const io = require "socket.io"

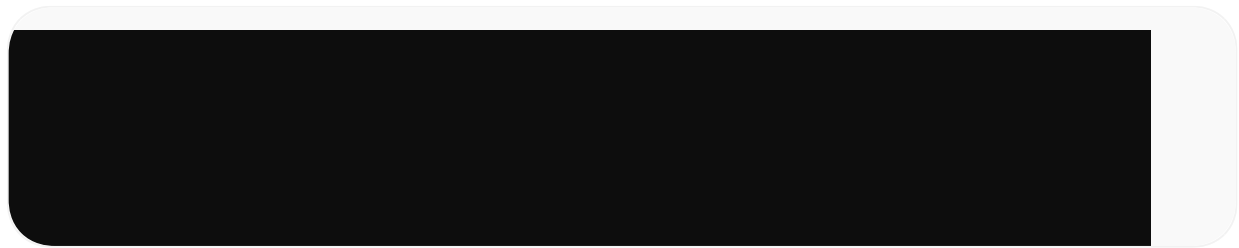
const app = express
const server = http.createServer(app)

const io = new Server(server)

server.listen(3000)
```

Here Socket.IO attaches to the **same HTTP server**.

Architecture becomes:



So one server handles both:

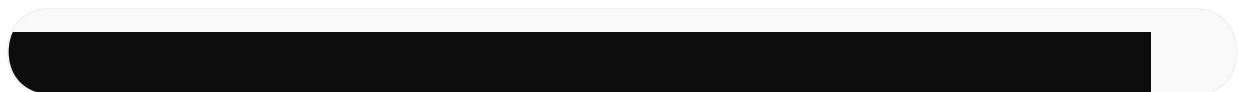
- HTTP requests
- WebSocket connections

4 What happens during WebSocket upgrade

When client connects:



Socket.IO intercepts the request:



Then it **upgrades** it to WebSocket:



Only the **HTTP server** can perform this upgrade.

That's why Socket.IO needs:

```
</> JavaScript
const server = http.createServer(app)
```

5 Visual Flow

Without Socket.IO:



With Socket.IO:



6 Important takeaway

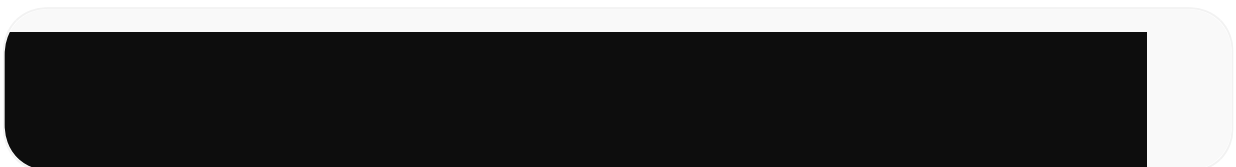
`express()` → request handler

`http.createServer(app)` → actual network server

Socket.IO attaches to the **real HTTP server**, not just Express.

✓ Simple analogy

Think of it like:



Both work inside the **same building (server)**.

💡 If you want, I can also explain a **very important hidden concept**:

Why `app.listen()` breaks Socket.IO in some setups and how it causes scaling issues.

This is something many Node developers don't realize.

explain me

Let's go deeper into why `app.listen()` can cause problems with Socket.IO and why most examples use `http.createServer(app)` instead.

1 What `app.listen()` actually does

When you write:

```
</> JavaScript
const app = express

app.listen(3000)
```

Express internally does this:

```
</> JavaScript
const http = require("http")

const server = http.createServer(app)
server.listen(3000)
```

So technically it still creates an HTTP server.

That's why Express works even if you don't manually create it.

2 The problem when using Socket.IO

Socket.IO must attach to the same HTTP server instance.

Correct way:

```
</> JavaScript
const express = require("express")
const http = require("http")
const io = require("socket.io")
```

```
const app = express
const server = http.createServer(app)

const io = new Server(server)

server.listen(3000)
```

Now both share the **same server**.

Architecture:



3 What beginners sometimes do (wrong)

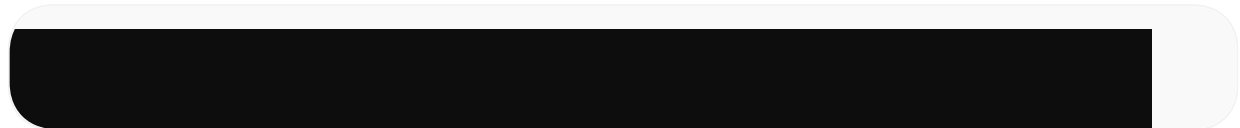
Many people write this:

JavaScript

```
const app = express
app.listen(3000)

const io = new Server(3000)
```

Now you accidentally created **two servers**.



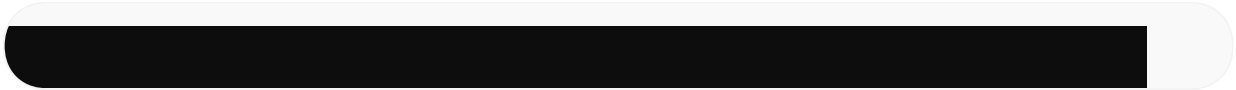
Problems:

- different ports
- cookies/session won't match
- authentication issues
- scaling issues
- CORS problems

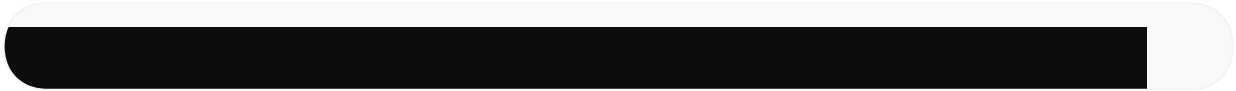
4 The biggest reason: WebSocket upgrade

Socket.IO starts as **HTTP polling** then upgrades to **WebSocket**.

Example request:



Then the server upgrades it:



Only the **actual HTTP server** can perform this upgrade.

So Socket.IO needs direct access to:

```
</> JavaScript
```

```
http.          app
```

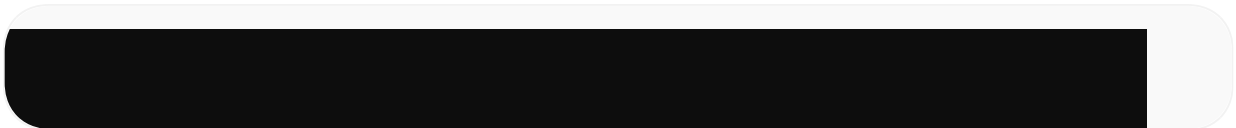
5 What happens internally

When you do:

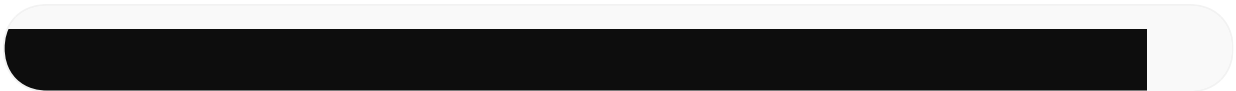
```
</> JavaScript
```

```
const server = http.          app  
const io = new Server server
```

Socket.IO attaches listeners to the server:



Specifically:



Flow:



6 Why professionals always use `createServer`

Because it gives **control over the server**.

Example:

JavaScript

```
server.  
server.  
server.  
server.
```

This is important for:

- WebSockets
- Socket.IO
- HTTPS
- clustering
- scaling
- load balancing

7 Real production architecture

Production apps usually run like this:



Both **Express + Socket.IO** share one server.

8 Clean professional setup

This is the **recommended structure**:

JavaScript

```
import express from "express"  
import http from "http"  
import Server from "socket.io"
```



```
const app = express
const server = http.createServer(app)

const io = new Server(server)

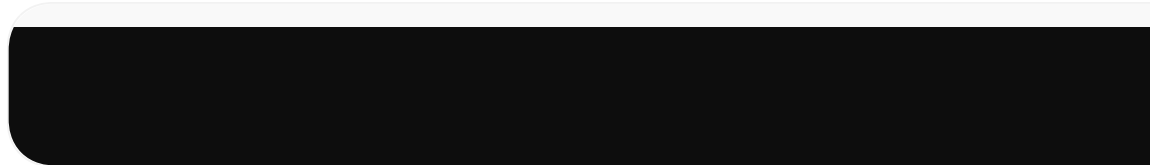
io.use((socket, next) => {
  // ...
})

app.get('/', (req, res) => {
  res.send("Hello World")
})

io.on("connection", (socket) => {
  console.log("User connected")
})

server.listen(3000, () => {
  console.log("Server running")
})
```

9 One line summary



🔥 If you're learning backend, the next concept you should understand is:

Why WebSockets require the `upgrade` event in Node.js and how Socket.IO hijacks it.

That concept makes Socket.IO architecture crystal clear.

can you explain me this docs

Broadcasting events
Socket.IO makes it easy to send events to all the connected clients.

info

Please note that broadcasting is a server-only feature.

To all connected clients

Broadcasting to all connected clients

```
io.emit("hello", "world");
```

caution

Clients that are currently disconnected (or in the process of reconnecting) won't receive the event. Storing this event somewhere (in a database, for example) is up to you, depending on your use case.

To all connected clients except the sender

Broadcasting to all connected clients excepting the sender

```
io.on("connection", (socket) => {  
  socket.broadcast.emit("hello", "world");  
});
```

note

In the example above, using `socket.emit("hello", "world")` (without broadcast flag) would send the event to "client A". You can find the list of all the ways to send an event in the cheatsheet.

With acknowledgements

Starting with Socket.IO 4.5.0, you can now broadcast an event to multiple clients and expect an acknowledgement from each one of them:

```
io.timeout(5000).emit("hello", "world", (err, responses)  
=> {  
  if (err) {  
    // some clients did not acknowledge the event in the  
    given delay  
  } else {  
    console.log(responses); // one response per client  
  }  
});
```

All broadcasting forms are supported:

in a room

```
io.to("room123").timeout(5000).emit("hello", "world",
```

```
(err, responses) => {
  // ...
});
```

from a specific socket

```
socket.broadcast.timeout(5000).emit("hello", "world",
(err, responses) => {
  // ...
});
```

in a namespace

```
io.of("/the-namespace").timeout(5000).emit("hello",
"world", (err, responses) => {
  // ...
});
```

With multiple Socket.IO servers

Broadcasting also works with multiple Socket.IO servers.

You just need to replace the default adapter by the Redis Adapter or another compatible adapter.

Broadcasting with Redis

In certain cases, you may want to only broadcast to clients that are connected to the current server. You can achieve this with the local flag:

```
io.local.emit("hello", "world");
```

Broadcasting with Redis but local

In order to target specific clients when broadcasting, please see the documentation about Rooms.

[Edit this page](#)

Socket.IO **broadcasting** means **sending an event from the server to multiple clients at once**.

Let's go through the docs section **line-by-line with examples** so it becomes clear.

1 What "Broadcasting events" means

Normally in Socket.IO you send messages like this:

```
</> JavaScript
```

```
socket.    "hello" "world"
```

This sends the event **only to that specific client**.

Broadcasting means:

```
</> JavaScript
```

```
send → many clients
```

Example:

```
</> JavaScript
```

```
io.    "hello" "world"
```

All connected clients receive:

```
</> JavaScript
```

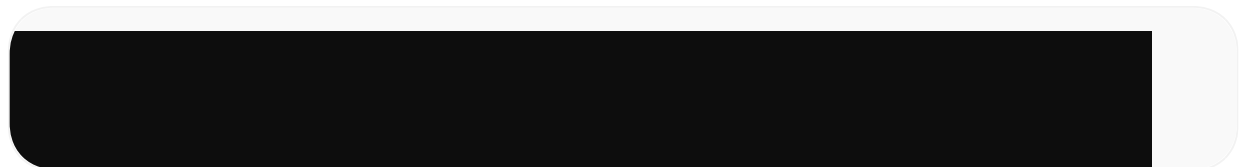
```
socket.    "hello"    msg  
console.    msg    // world
```

2 Broadcasting to all connected clients

```
</> JavaScript
```

```
io.    "hello" "world"
```

What happens



Server sends:



Result:

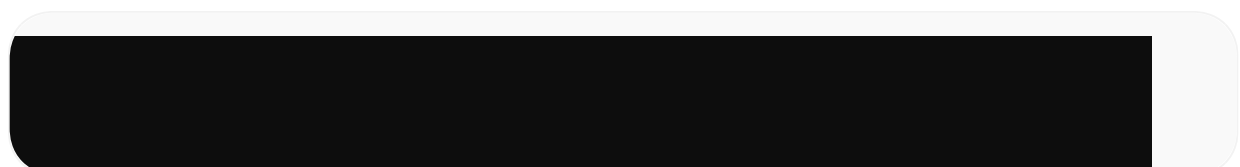


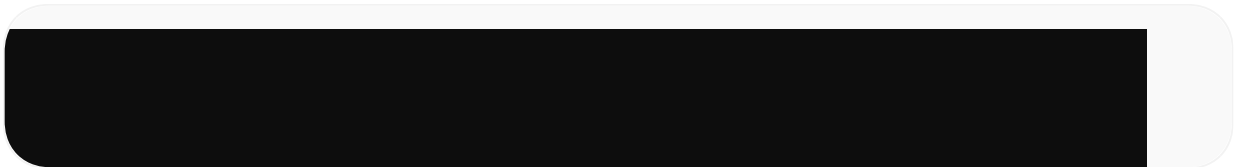
Diagram:



3 Important note in docs

Clients that are currently disconnected won't receive the event.

Example:

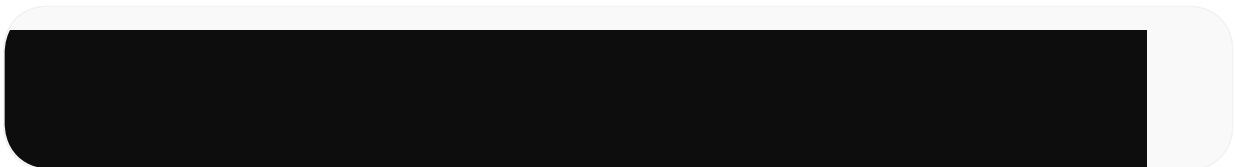


Server runs:

JavaScript

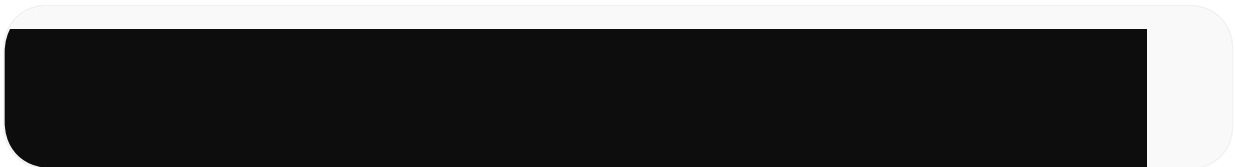
```
io.      "new-message" "hello"
```

Result:



Socket.IO **does not store messages.**

If you want persistence, you must store it yourself:



Example:

JavaScript

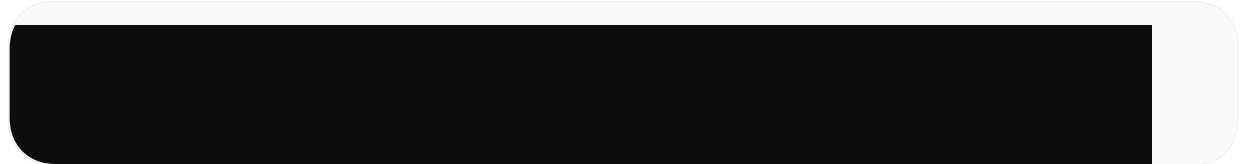
```
await db.      message  
io.      "new-message" message
```

4 Broadcasting to everyone EXCEPT the sender

Docs example:

```
</> JavaScript
io.  "connection"  socket
socket.  .  "hello" "world"
```

Assume this scenario:



Code:

```
</> JavaScript
socket.  .  "new-message" msg
```

Result:

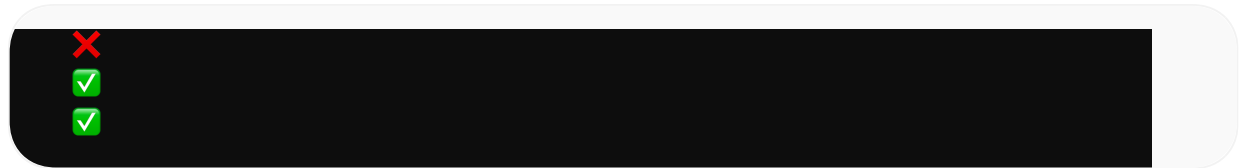
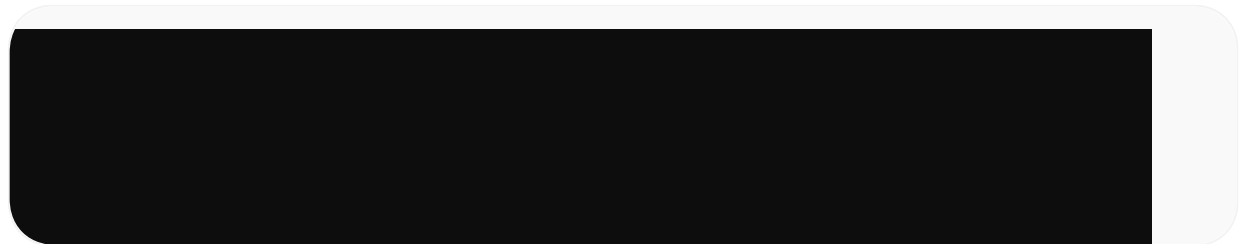


Diagram:



Difference

Method	Who receives
<code>socket.emit()</code>	only sender
<code>socket.broadcast. emit()</code>	everyone except sender

Method	Who receives
<code>io.emit()</code>	everyone including sender

5 Broadcasting with acknowledgements

This is a **new feature (Socket.IO 4.5+)**.

Normally you don't know if clients received the message.

Now you can require a **confirmation (ack)**.

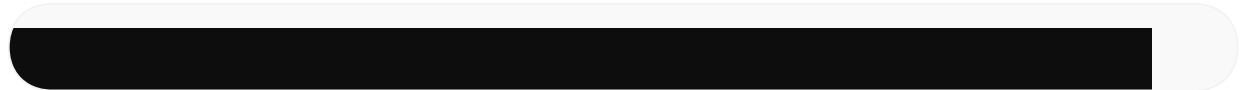
Example:

JavaScript

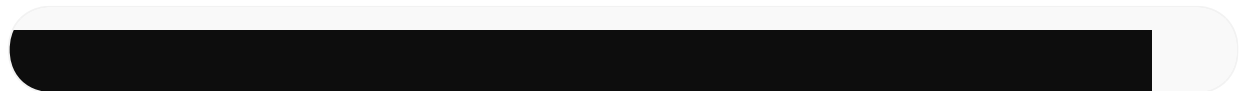
```
io.emit(5000, "hello", "world", err, responses)
```

What happens

Server sends event:



Each client must reply:



Example client:

JavaScript

```
socket.emit("hello", msg, callback)
console.log(msg)
callback("received")
```

Server receives:

JavaScript

```
responses = ["received", "received", "received"]
```

If a client doesn't respond within **5 seconds**:

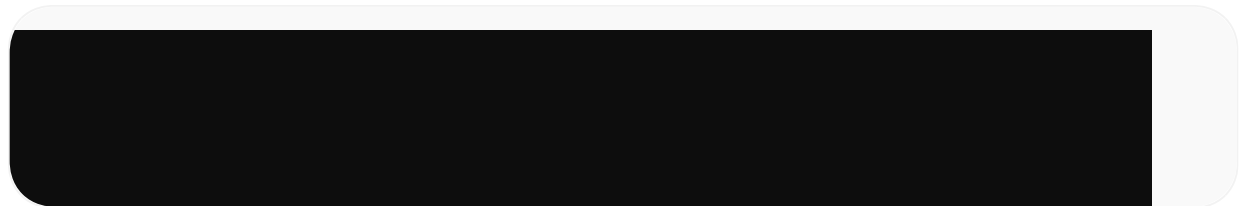
```
</> JavaScript
```

```
err = true
```

6 Broadcasting inside a room

Rooms are groups of clients.

Example:

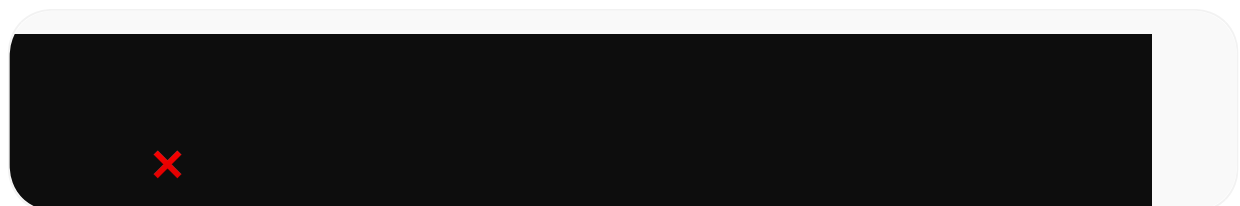


Broadcast:

```
</> JavaScript
```

```
io. "room123" . "hello" "world"
```

Only room members receive it.



With acknowledgement:

```
</> JavaScript
```

```
io. "room123"  
  . 5000  
  . "hello" "world" err responses
```

7 Broadcasting from a specific socket

Example:

```
</> JavaScript
```

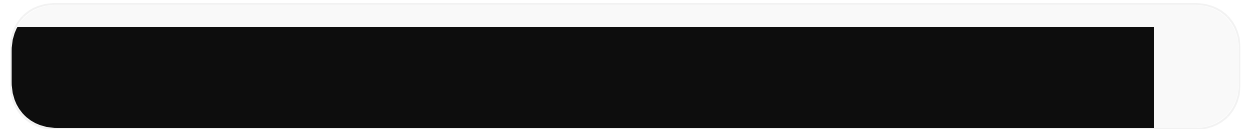
```
socket. . "hello" "world"
```

But with acknowledgement:


```
</> JavaScript
```

```
socket.  
  .      5000  
  .      "hello" "world" err responses
```

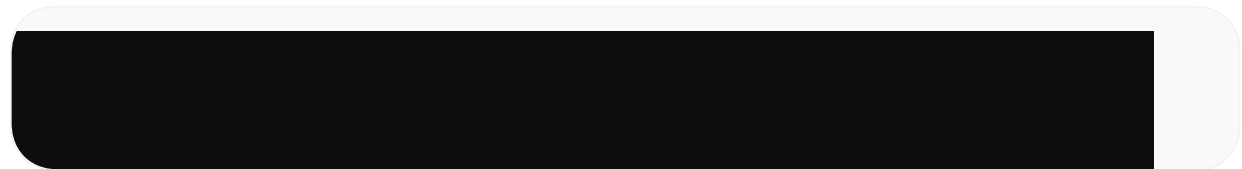
Meaning:



8 Broadcasting in a namespace

Namespaces split your app into channels.

Example:

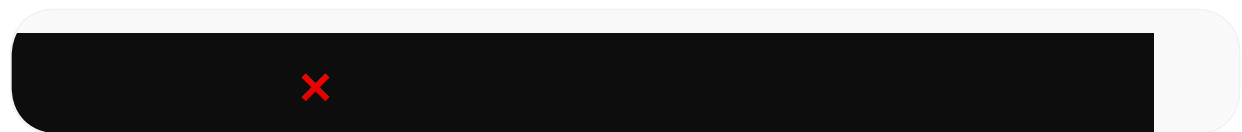


Broadcast:

```
</> JavaScript
```

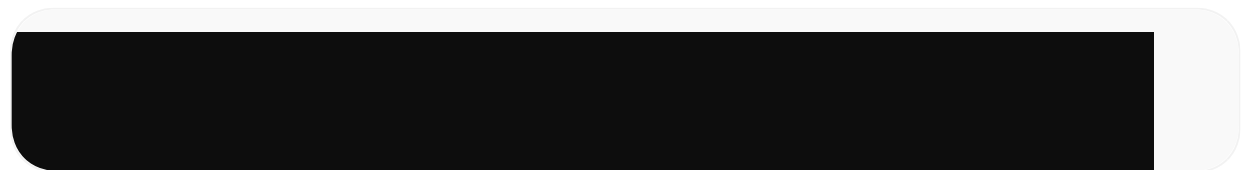
```
io.  "/chat" .  "hello" "world"
```

Only `/chat` clients receive it.



9 Broadcasting with multiple servers

In production you often run multiple Node servers:



Each server has its own connected clients.

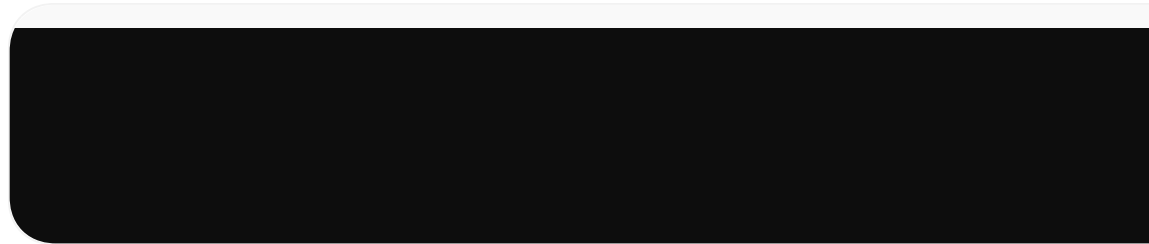
Without Redis:

Clients on other servers **won't receive the event.**

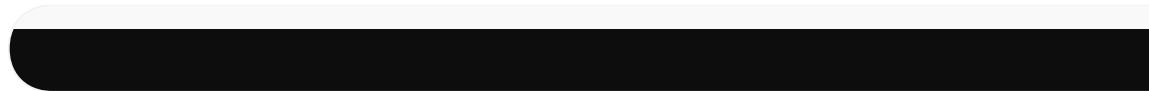
Solution → Redis Adapter

Redis synchronizes events across servers.

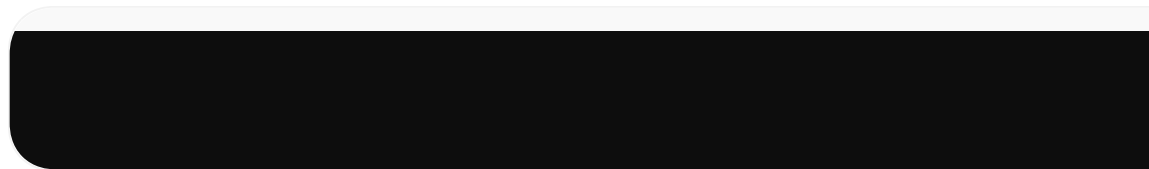
Architecture:



Now:



Result:



All receive the event.

10 Local broadcast

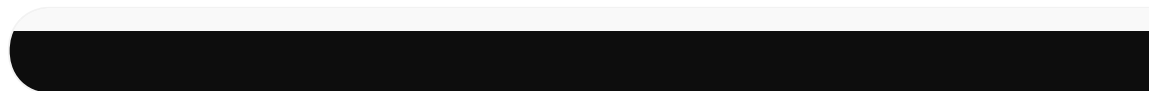
Sometimes you **don't want Redis broadcast.**

Docs example:

```
<> JavaScript
```

```
io.      .    "hello" "world"
```

Meaning:



Useful when:

1 1 Quick cheat sheet

Method	Meaning
<code>socket.emit()</code>	send to one client
<code>socket.broadcast.emit()</code>	send to everyone except sender
<code>io.emit()</code>	send to all clients
<code>io.to(room).emit()</code>	send to a room
<code>io.of(namespace).emit()</code>	send to namespace
<code>io.timeout().emit()</code>	expect acknowledgement
<code>io.local.emit()</code>	broadcast only on this server

Real world example (chat app)

User sends message:

JavaScript

```
socket.  "send-message"  msg
socket.  .  "receive-message"  msg
```

Result:

💡 If you'd like, I can also explain the **4 most important Socket.IO concepts developers must understand**:

- 1 Rooms
- 2 Namespaces
- 3 Adapters (Redis scaling)
- 4 Connection lifecycle

These are **core concepts used in real-time apps like WhatsApp, Discord, and multiplayer games**.

explain me

I'll explain the **4 most important Socket.IO concepts** clearly. These are the concepts used to build **real apps like chat apps, multiplayer games, collaboration tools, etc.**

1 Rooms (Group clients together)

A **room** is a group of sockets.

Example use cases:

- Chat rooms
- Game lobbies
- Private groups
- Notifications for specific users

Join a room

JavaScript

```
io. "connection" socket
socket. "room1"
```

Now that socket belongs to **room1**.

Send message to the room

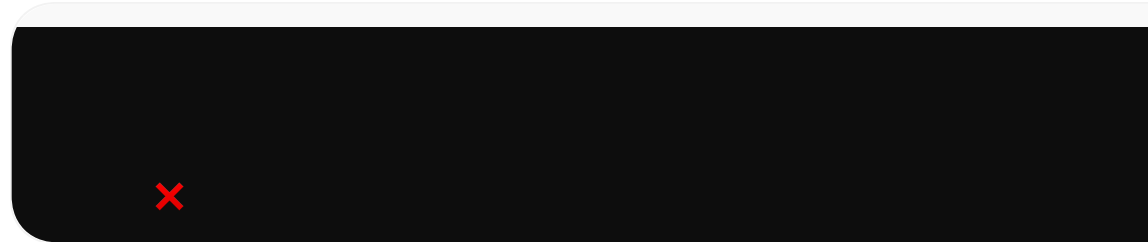
```
</> JavaScript
```

```
io.    "room1" .    "message" "Hello room1"
```

Only clients inside **room1** receive it.



Result:



Example: Chat room

Server:

```
</> JavaScript
```

```
io.    "connection" socket
socket.    "chat-room"

socket.    "send-message" msg
io.    "chat-room" .    "receive-message" msg
```

Client:

```
</> JavaScript
```

```
socket.    "receive-message" msg
console.    msg
```

2 Namespaces (Separate applications)

Namespaces divide your socket server into **multiple channels**.

Default namespace:

Example namespaces:

Create namespace

```
</> JavaScript
```

```
const chat = io.  "/chat"
```

Connection:

```
</> JavaScript
```

```
chat.  "connection"  socket  
  console.  "User joined chat namespace"
```

Client connects:

```
</> JavaScript
```

```
const socket = io  "/chat"
```

Architecture

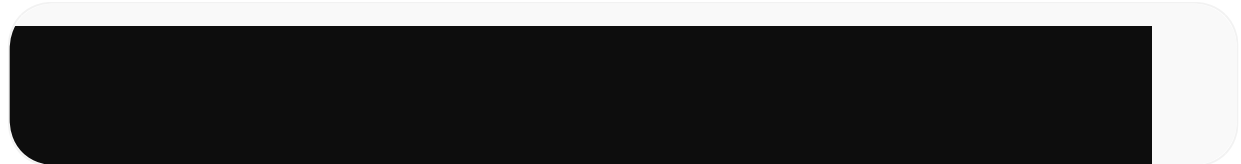
Real world example

Each namespace has **separate events**.

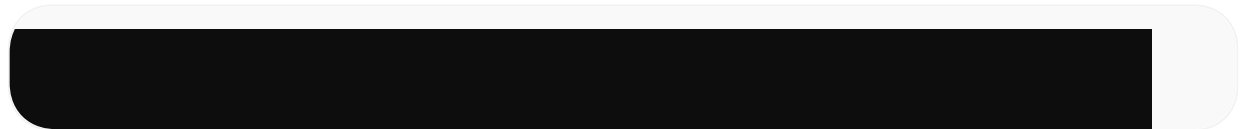
3 Adapters (Scaling Socket.IO)

When your app grows, you may run **multiple servers**.

Example:



Problem:



If Server1 runs:

```
</> JavaScript
```

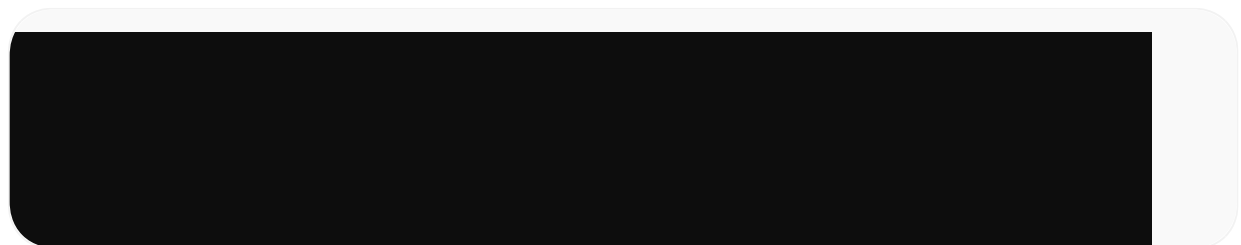
```
io.      "message" "hello"
```

User B **won't** receive it.

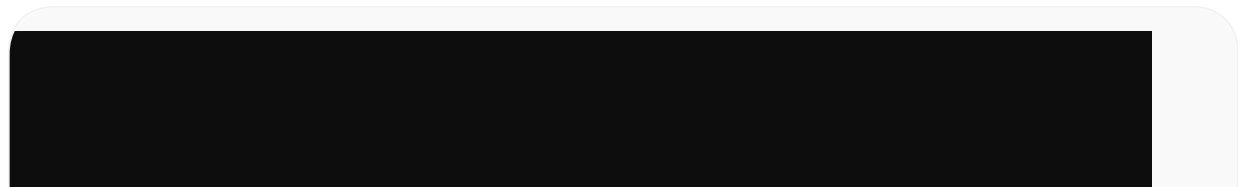
Solution → Redis adapter

Redis shares events between servers.

Architecture:



Now:



All clients receive the event.

Install Redis adapter

❏ Bash

```
npm
```

Example:

❏ JavaScript

```
import createAdapter from "@socket.io/redis-adapter"
import createClient from "redis"

const pubClient = createClient
const subClient = pubClient.

io.      createAdapter pubClient subClient
```

Now Socket.IO works across servers.

4 Connection Lifecycle

Socket.IO connections go through **multiple stages**.

1. Client connects

Client:

❏ JavaScript

```
const socket = io "http://localhost:3000"
```

Server:

❏ JavaScript

```
io.  "connection" socket
console.  "User connected"
```

2. Communication

Client sends event:

```
</> JavaScript
socket.  "message"  "Hello server"
```

Server receives:

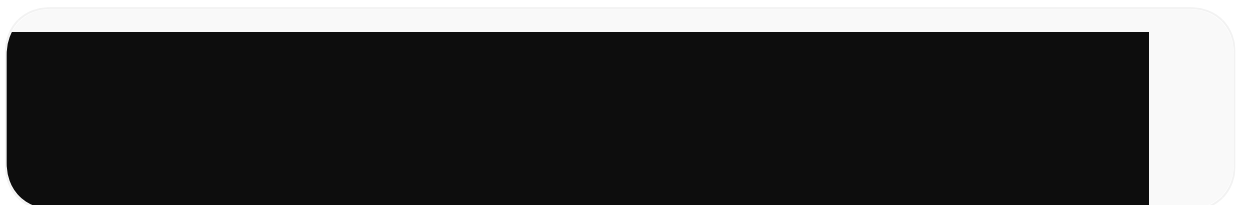
```
</> JavaScript
socket.  "message"  msg
console.  msg
```

3. Disconnection

Server:

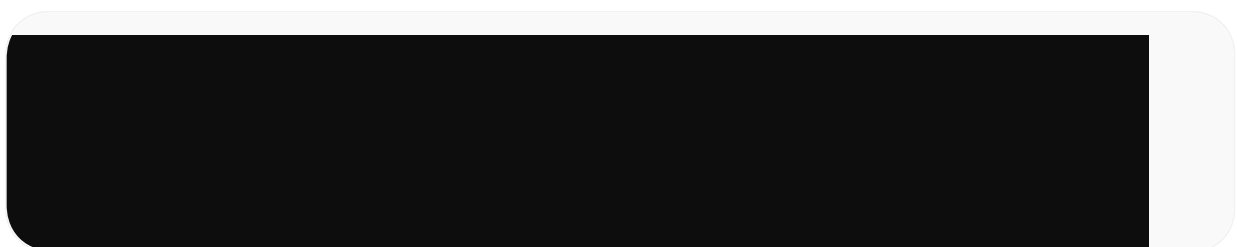
```
</> JavaScript
socket.  "disconnect"
console.  "User disconnected"
```

Possible reasons:



4. Reconnection (automatic)

Socket.IO automatically reconnects.

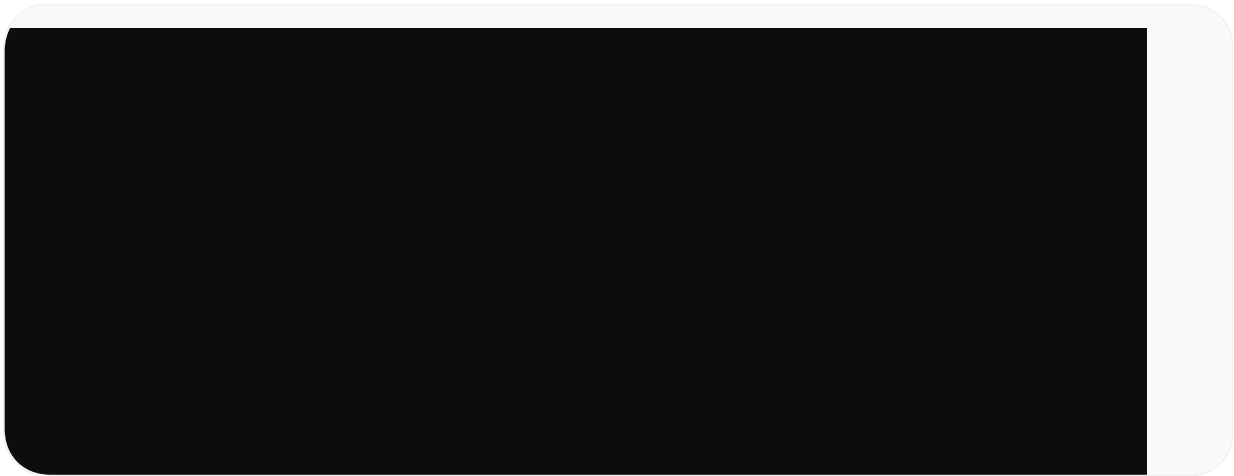


Client events:

```
</> JavaScript
socket.  "reconnect"
console.  "Reconnected"
```

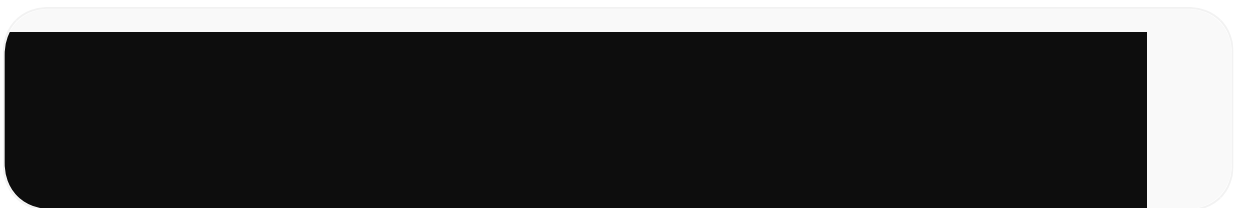


Full lifecycle diagram

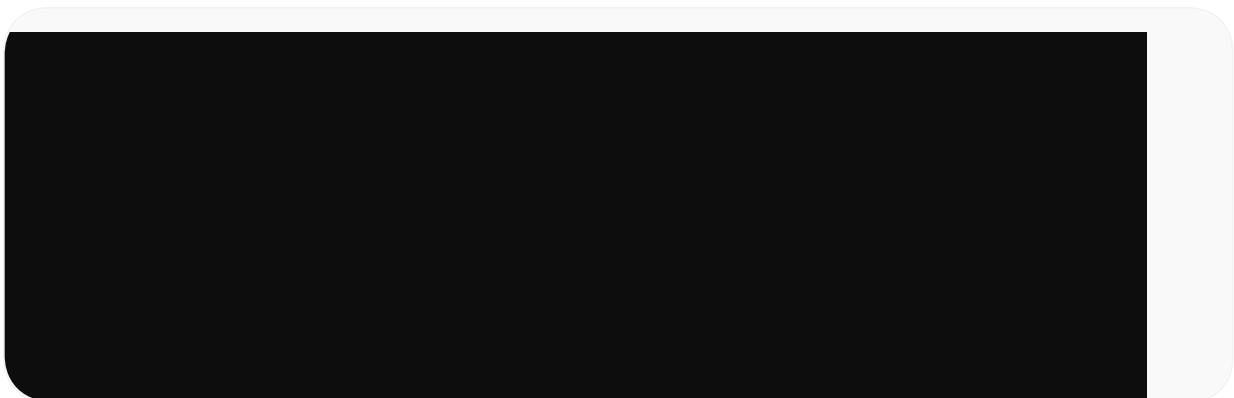


How these concepts work together

Example: Chat application



Architecture:



Summary

Concept	Purpose
Rooms	group users

Concept	Purpose
Namespaces	separate channels
Adapters	scale across servers
Lifecycle	manage connection state

💡 If you're learning Socket.IO seriously, the **next 3 advanced concepts** will make everything click:

- 1 How Socket.IO actually works internally (polling → WebSocket upgrade)
- 2 Difference between `socket`, `io`, and `server` objects
- 3 How WhatsApp/Discord style real-time systems are designed

These are the things **most developers never fully understand**, but they make Socket.IO much easier to master.

explain

Let's go deeper into the **3 advanced Socket.IO concepts** that make everything click. Once you understand these, Socket.IO architecture becomes much easier to reason about. 🚀

1 How Socket.IO actually works (Polling → WebSocket upgrade)

Socket.IO **does not immediately start with WebSocket**.

It starts with **HTTP long polling** and then upgrades to WebSocket if possible.

Step 1 — Initial HTTP request

When the client connects:

```
<> JavaScript
```

```
const socket = io "http://localhost:3000"
```

The browser first makes a normal HTTP request:

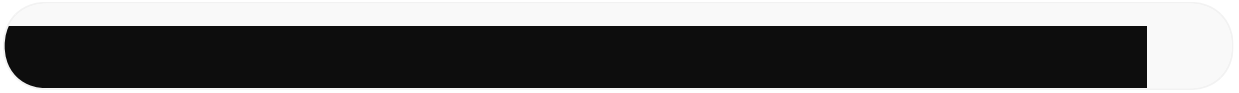
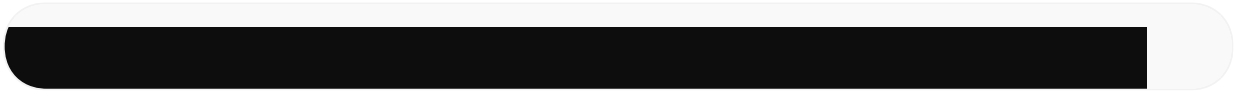


Diagram:



This ensures the connection works even if WebSockets are blocked.

Step 2 — Server responds with session info

Server sends something like:



Now both sides know the **session ID**.

Step 3 — Upgrade attempt

Client tries to upgrade the connection:

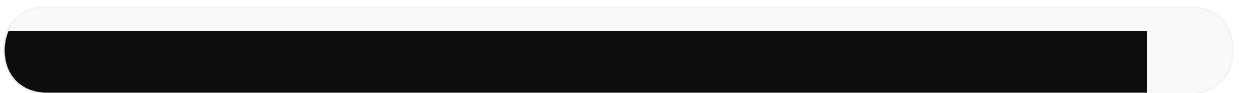
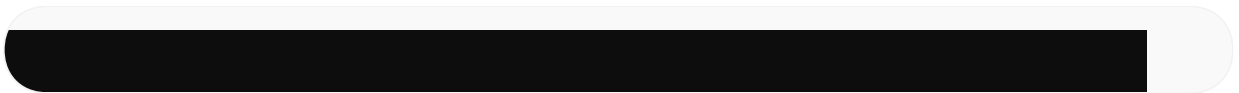


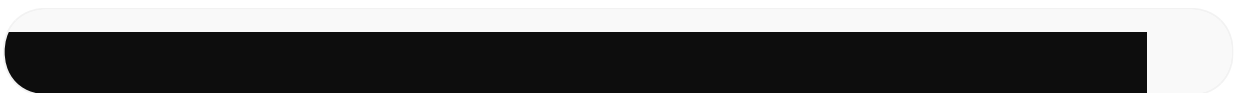
Diagram:



If successful:



Final architecture:

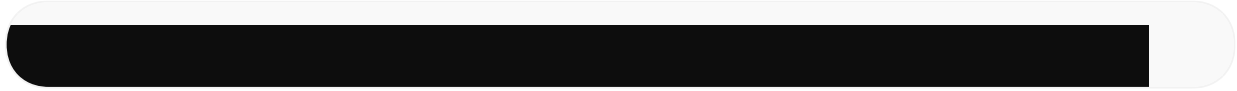


WebSocket is faster because it's **persistent and full-duplex**.

Why this fallback exists

Some networks block WebSockets (corporate proxies, old browsers).

So Socket.IO guarantees connection by using:



2 Difference between `io`, `socket`, and `server`

This is one of the most confusing things for beginners.

`server`

This is the **Node HTTP server**.

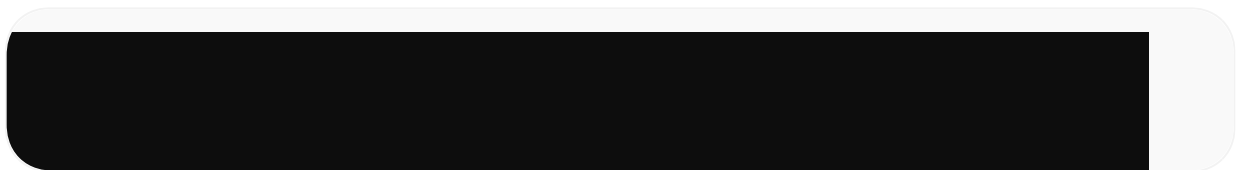
Example:

```
JavaScript
const server = http.createServer(app)
```

Responsibilities:

- Handles HTTP requests
- Handles WebSocket upgrade
- Runs on a port

Diagram:



`io`

`io` is the **Socket.IO server instance**.

```
JavaScript
const io = new Server(server)
```

Responsibilities:

- manages all sockets

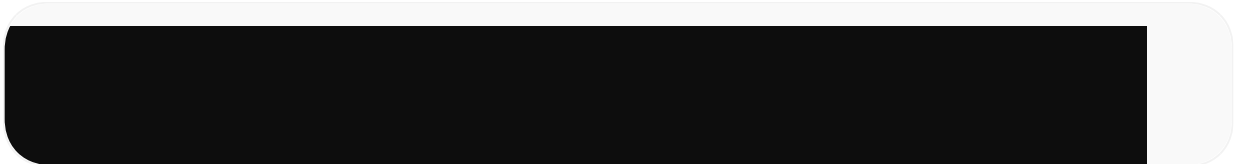
- broadcasting
- namespaces
- rooms

Example:

JavaScript

```
io.      "message" "hello"
```

Diagram:



socket

socket represents **one connected client**.

JavaScript

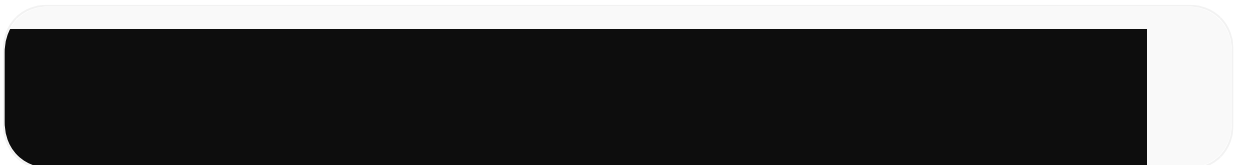
```
io.      "connection" socket  
console. socket.
```

Example:

JavaScript

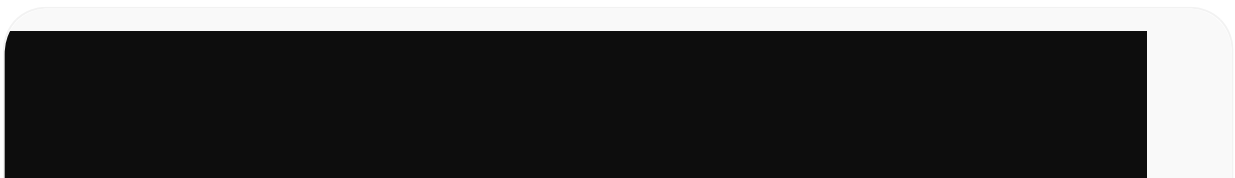
```
socket.      "message" "hello"
```

Diagram:



Each connection gets a **unique socket object**.

Full relationship



Example showing all three

</> JavaScript

```
import express from "express"
import http from "http"
import Server from "socket.io"

const app = express

const server = http.createServer(app) // HTTP server

const io = new Server(server) // socket.io server

io.on("connection", (socket) => { // individual client
  console.log("User connected:", socket.id)

  socket.emit("welcome", "hello user")
})

server.listen(3000)
```

3 How real-time apps like WhatsApp / Discord are designed

Real-time apps combine multiple Socket.IO features together.

Typical architecture:



What each component does

Load balancer

- distributes users across servers

Multiple servers

- handle thousands of connections

Redis adapter

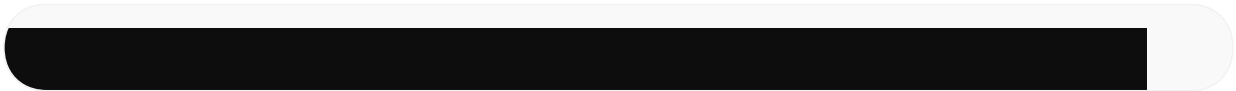
- syncs events between servers

Database

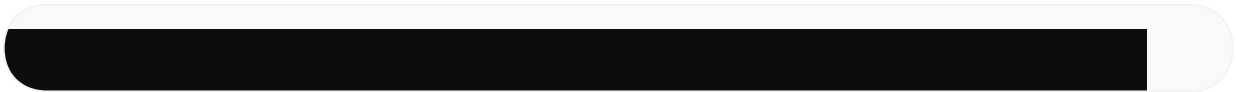
- stores messages permanently

Message flow example (chat app)

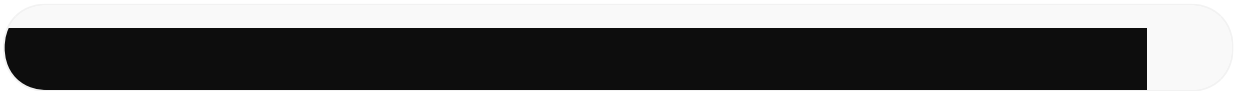
User A sends message:



Server1 saves message:



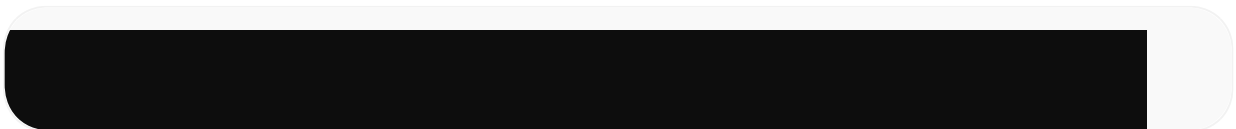
Server1 broadcasts event:



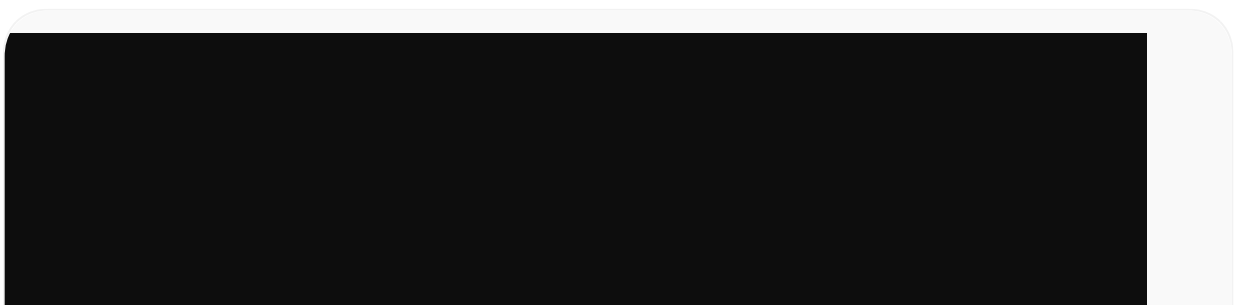
Redis forwards to other servers:



Servers send message to clients:



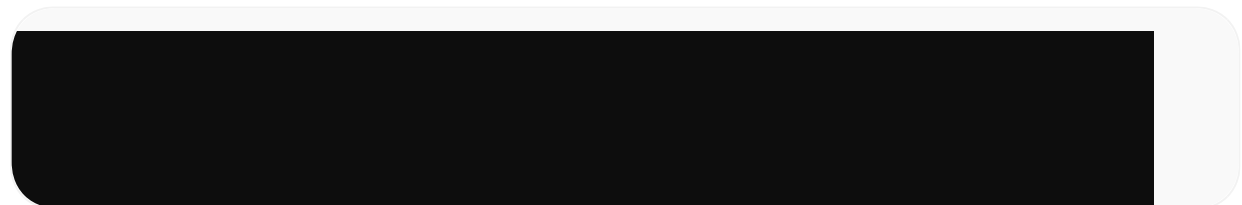
Final flow:



How rooms are used in chat apps

Each chat conversation becomes a **room**.

Example:



Broadcast:

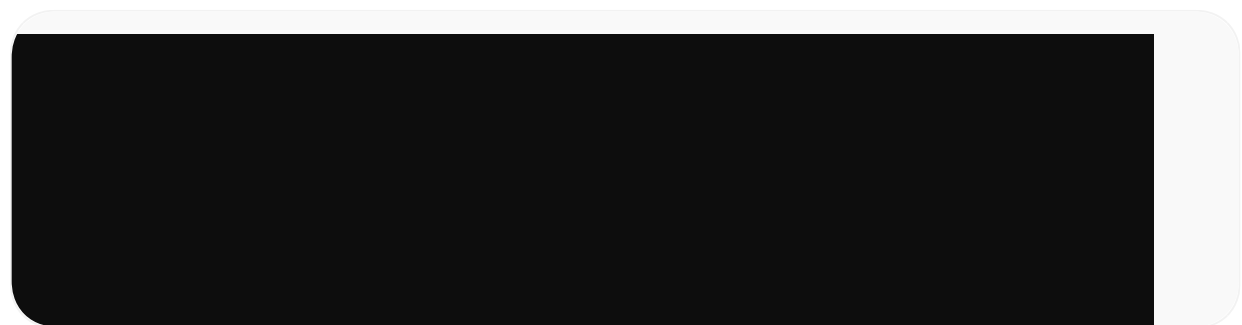
```
</> JavaScript
```

```
io. "chat123" . "new-message" message
```

Only participants receive the message.

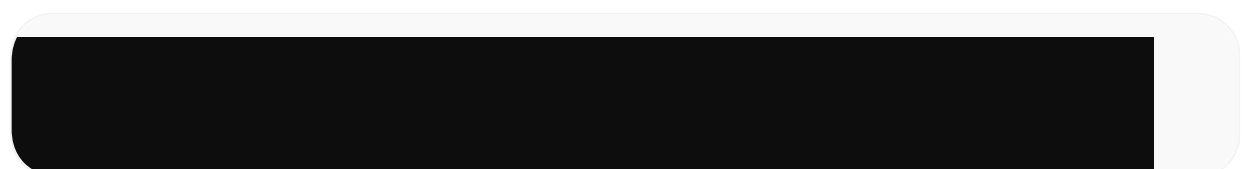
The key idea

Socket.IO real-time systems combine:



All working together.

One-line mental model



If you'd like, I can also explain **one extremely important concept most Socket.IO developers misunderstand**:

Why Socket.IO apps sometimes randomly lose messages or duplicate messages in production and how companies like Discord prevent it.

This is a **very important real-time system design topic**.

what redis actually dose ?

What Redis Actually Does

At its core, **Redis is a very fast in-memory data store**.

It stores data **in RAM instead of disk**, which makes it extremely fast ⚡.

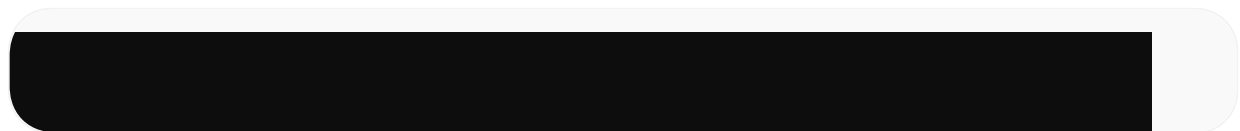
Think of Redis as a **super-fast temporary database + message broker + cache**.

1 Basic Idea

Normal databases like **MySQL** or **MongoDB** store data on disk.

Disk access is slower.

Redis stores data in **RAM**:



So Redis is used when speed matters.

2 What Redis Is Used For

Redis usually serves **four main purposes**.

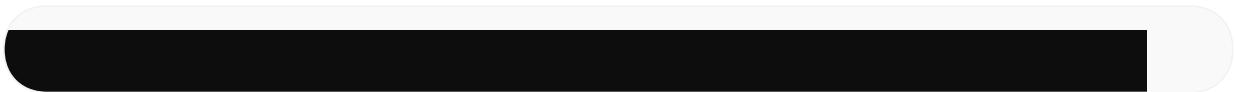
Purpose	Example
Caching	speed up APIs
Pub/Sub messaging	real-time systems
Session storage	login sessions

Purpose	Example
Queues / background jobs	task processing

3 Redis as a Cache (Most Common Use)

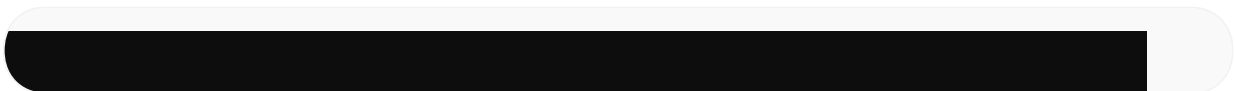
Suppose your API fetches data from a database.

Without Redis:

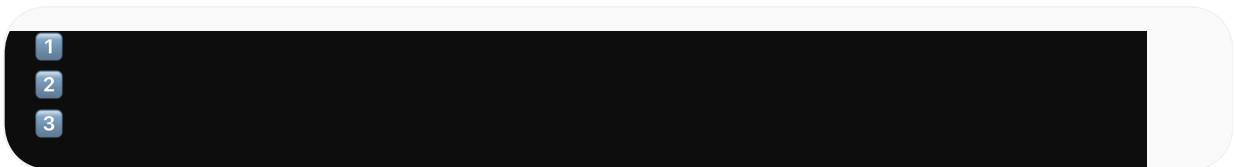


Database queries are expensive.

With Redis caching:



Flow:



Example:

`</> JavaScript`

```
const cachedUser = await redis.get("user:123")

if (cachedUser) {
  return JSON.parse(cachedUser)
}

const user = await db.query(`SELECT * FROM users WHERE id = 123`)
await redis.set("user:123", JSON.stringify(user))

return user
```

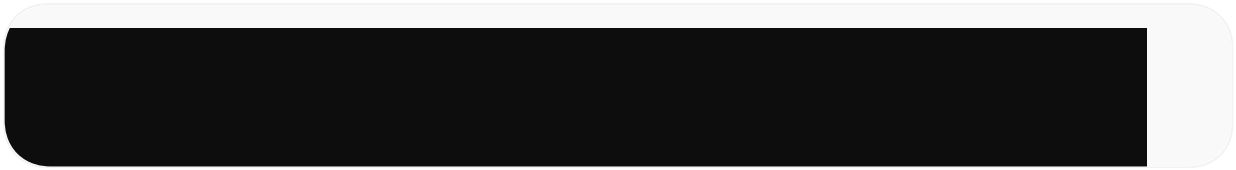
This can make APIs 10–100x faster 🚀.

4 Redis Pub/Sub (Used by Socket.IO)

Redis also acts as a **message broker**.

Servers can **publish messages**, and others **subscribe**.

Diagram:



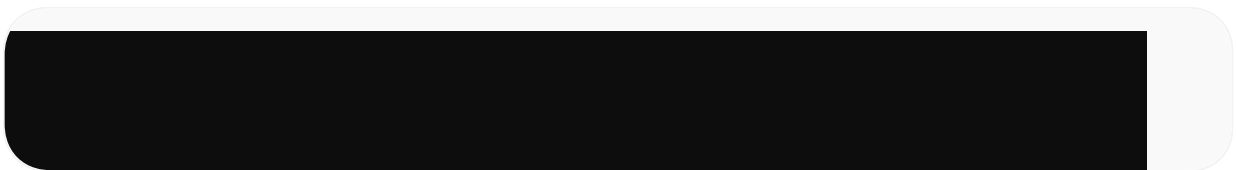
Example:



This is why Redis is used with Socket.IO.

Example with Socket.IO

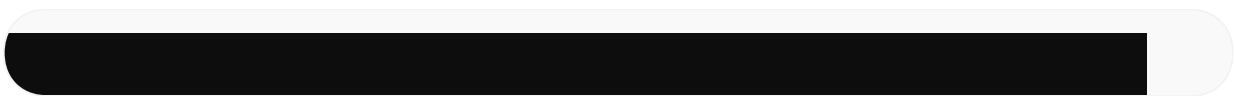
Multiple servers:



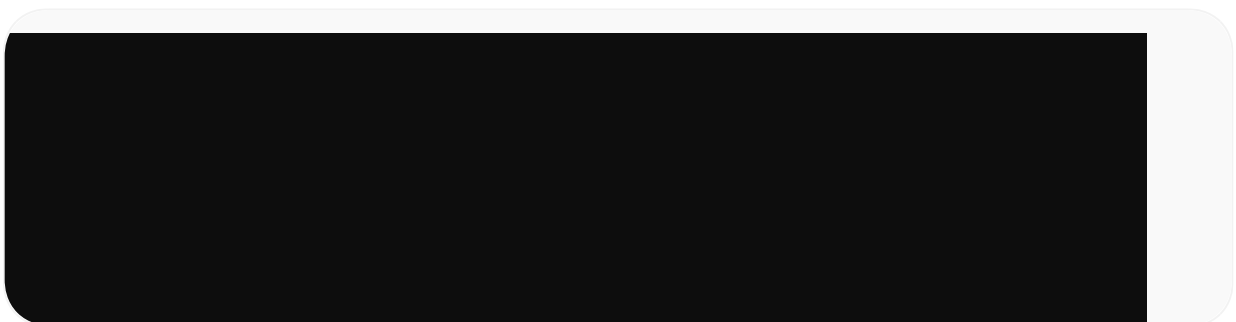
User A connected to Server1

User B connected to Server2

Without Redis:



With Redis adapter:



Redis synchronizes events across servers.

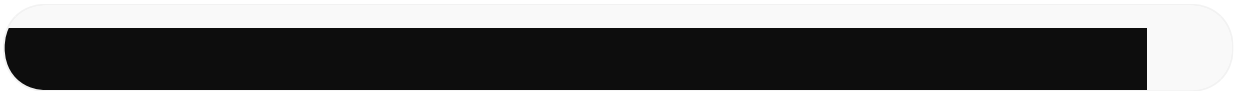
5 Redis Data Types

Redis is not just key-value storage.

It supports special structures.

Type	Example
String	"user:1" → "John"
List	message queue
Set	unique items
Hash	objects
Sorted Set	rankings

Example:



Stored as a **hash**.

6 Redis Is Extremely Fast

Typical speeds:



Because everything is in RAM.

7 Example Real Architecture

Real-time chat app architecture:





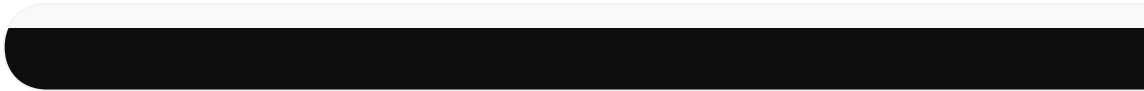
Roles:

Component	Job
Servers	handle connections
Redis	sync events
Database	permanent storage

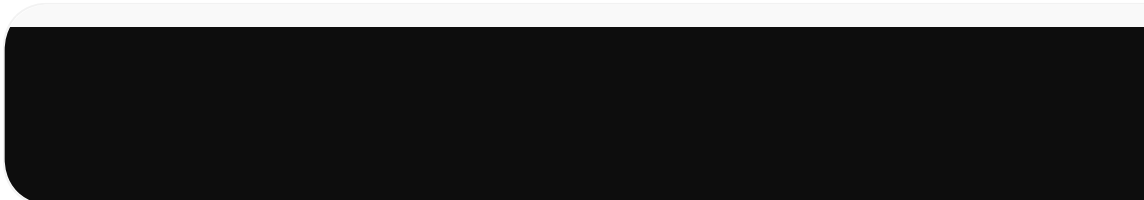
8 Important: Redis Is Usually Not Permanent Storage

Redis is not usually your main database.

Because:



So Redis is used for:

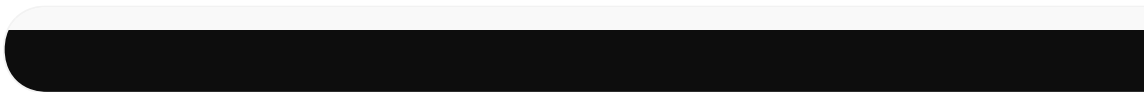


Permanent data goes to:

- PostgreSQL
- MongoDB

9 One-Sentence Explanation

Redis is:



✓ Mental model

💡 If you're working with **Node.js + Socket.IO**, the next thing worth understanding is:

Why Redis adapters are required only when you have multiple servers and how load balancers break WebSocket connections without sticky sessions.

That concept explains **how real-time systems scale to millions of users.**

now explain me this docs

Adapter

An Adapter is a server-side component which is responsible for broadcasting events to all or a subset of clients.

When scaling to multiple Socket.IO servers, you will need to replace the default in-memory adapter by another implementation, so the events are properly routed to all clients.

Here is the list of adapters that are maintained by our team:

- the Redis adapter
- the Redis Streams adapter
- the MongoDB adapter
- the Postgres adapter
- the Cluster adapter
- the Google Cloud Pub/Sub adapter
- the AWS SQS adapter
- the Azure Service Bus adapter

There are also several other options which are maintained by the (awesome!) community:

AMQP (e.g. RabbitMQ)

NATS

NATS

Please note that enabling sticky sessions is still needed

when using multiple Socket.IO servers and HTTP long-polling. More information [here](#).

API

You can have access to the adapter instance with:

```
// main namespace
const mainAdapter = io.of("/").adapter; // WARNING!
io.adapter() will not work
// custom namespace
const adminAdapter = io.of("/admin").adapter;
```

Starting with socket.io@3.1.0, each Adapter instance emits the following events:

create-room (argument: room)
delete-room (argument: room)
join-room (argument: room, id)
leave-room (argument: room, id)

Example:

```
io.of("/").adapter.on("create-room", (room) => {
  console.log( room ${room} was created );
});
```

```
io.of("/").adapter.on("join-room", (room, id) => {
  console.log( socket ${id} has joined room
${room} );
});
```

Emitter

Most adapter implementations come with their associated emitter package, which allows communicating to the group of Socket.IO servers from another Node.js process.

Emitter diagram

This may be useful for example in a microservice setup, where all clients connect to the microservice M1, while the microservice M2 uses the emitter to broadcast packets (uni-directional communication).

Emitter cheatsheet

```
// to all clients
```



```
emitter.emit(/* ... */);

// to all clients in "room1"
emitter.to("room1").emit(/* ... */);

// to all clients in "room1" except those in "room2"
emitter.to("room1").except("room2").emit(/* ... */);

const adminEmitter = emitter.of("/admin");

// to all clients in the "admin" namespace
adminEmitter.emit(/* ... */);

// to all clients in the "admin" namespace and in the
"room1" room
adminEmitter.to("room1").emit(/* ... */);

The emitter also supports the utility methods that were
added in socket.io@4.0.0:

socketsJoin()
// make all Socket instances join the "room1" room
emitter.socketsJoin("room1");

// make all Socket instances of the "admin" namespace in
the "room1" room join the "room2" room
emitter.of("/admin").in("room1").socketsJoin("room2");

socketsLeave()
// make all Socket instances leave the "room1" room
emitter.socketsLeave("room1");

// make all Socket instances in the "room1" room leave
the "room2" and "room3" rooms
emitter.in("room1").socketsLeave(["room2", "room3"]);

// make all Socket instances in the "room1" room of the
"admin" namespace leave the "room2" room
emitter.of("/admin").in("room1").socketsLeave("room2");

disconnectSockets()
// make all Socket instances disconnect
emitter.disconnectSockets();
```

```
// make all Socket instances in the "room1" room
disconnect (and discard the low-level connection)
emitter.in("room1").disconnectSockets(true);

// make all Socket instances in the "room1" room of the
"admin" namespace disconnect
emitter.of("/admin").in("room1").disconnectSockets();

// this also works with a single socket ID
emitter.of("/admin").in(theSocketId).disconnectSockets()
;

serverSideEmit()
// emit an event to all the Socket.IO servers of the cluster
emitter.serverSideEmit("hello", "world");

// Socket.IO server (server-side)
io.on("hello", (arg) => {
  console.log(arg); // prints "world"
});
```

This documentation is explaining **how Socket.IO works internally when you scale your app to multiple servers**.

The key concept here is the **Adapter**.

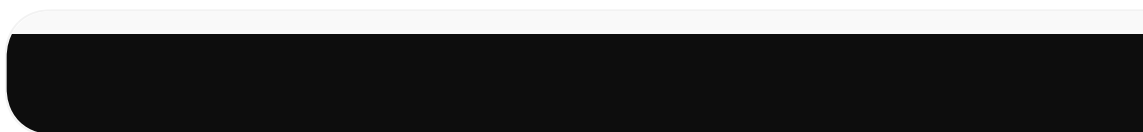
I'll explain it step-by-step so the architecture becomes clear. 🧠

1 What an Adapter is

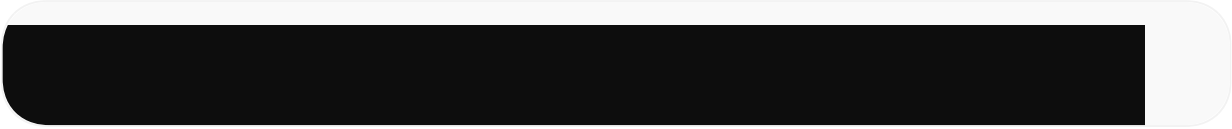
In Socket.IO, an **Adapter** is the component that manages:

- rooms
- sockets
- broadcasting events

Inside a single server, the adapter keeps track of:



Example internal structure:

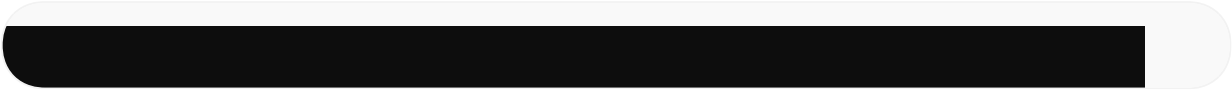


So when you do:

```
</> JavaScript
```

```
io.    "room1" .    "message" "hello"
```

The adapter finds:



Then sends the event to those sockets.

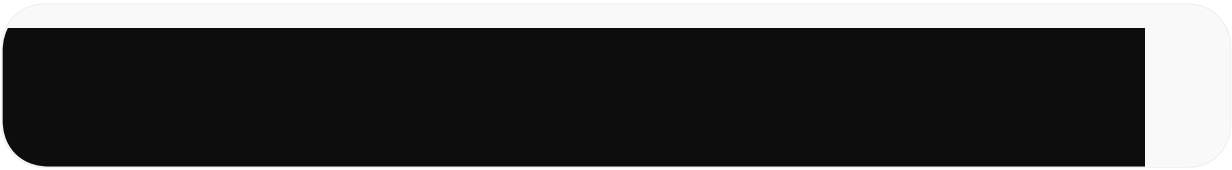
2 Default Adapter (Single Server)

When you run one server:



The default adapter is **in-memory**.

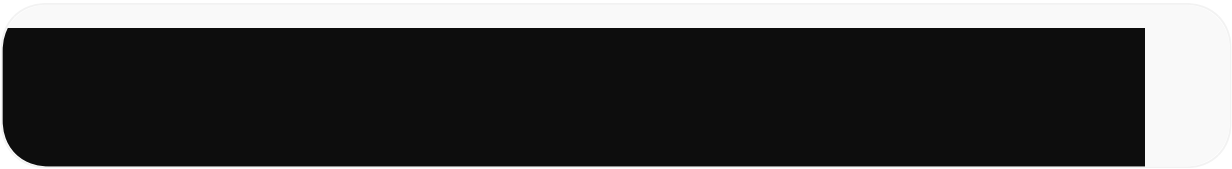
Meaning the room data lives inside the server memory:



This works fine for **one server**.

3 Problem When Scaling Servers

Suppose you scale your app:



Clients connect randomly:

Now if Server1 broadcasts:

```
</> JavaScript
```

```
io.    "room1" .    "message" "hello"
```

Only clients connected to **Server1** receive it.

Because Server1 **does not know about sockets on Server2 or Server3**.

4 Adapter Solution

Adapters solve this by using a **shared messaging system**.

Example using **Redis**:

When Server1 broadcasts:

All servers receive the event and send it to their clients.

Now broadcasting works globally.

5 Official Adapters

The docs list several adapters because different infrastructures use different message systems.

Examples:

Adapter	Uses
Redis adapter	most common

Adapter	Uses
MongoDB adapter	uses Mongo change streams
Postgres adapter	uses Postgres pub/sub
Cluster adapter	multiple Node workers
Google Cloud Pub/Sub	Google cloud apps
AWS SQS	AWS architecture
Azure Service Bus	Azure apps

Community adapters include:

- **RabbitMQ**
- **NATS**

All of them do the same job:

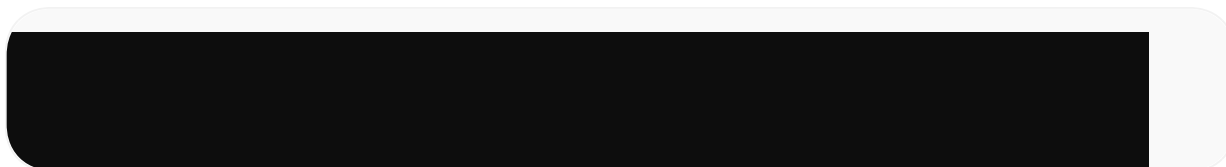


6 Sticky Sessions (Important)

Docs mention **sticky sessions**.

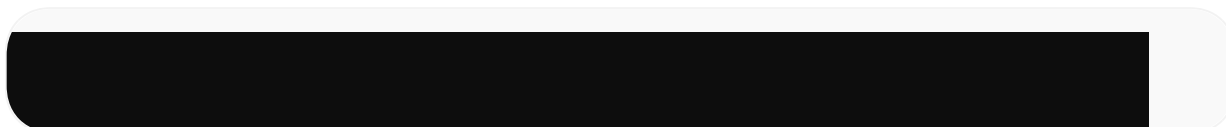
This is required because Socket.IO may use **HTTP polling**.

Example without sticky sessions:



But polling requires **same server**.

Sticky session ensures:



Load balancer maintains this.

7 Accessing the Adapter

You can access the adapter instance:

```
</> JavaScript
```

```
const mainAdapter = io.  "/" .
```

or for a namespace:

```
</> JavaScript
```

```
const adminAdapter = io.  "/admin" .
```

This gives you access to **room lifecycle events**.

8 Adapter Events

Adapters emit events when rooms change.

create-room

Triggered when a new room is created.

```
</> JavaScript
```

```
io.  "/" .  "create-room"  room  
  console.  `room  room  created`
```

Example:

Output:

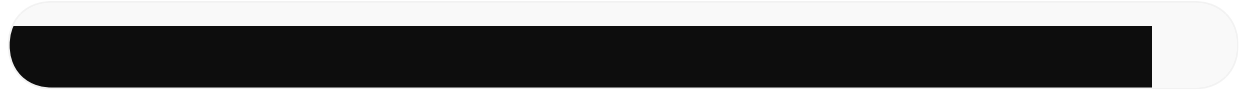
join-room

Triggered when a socket joins.

</> JavaScript

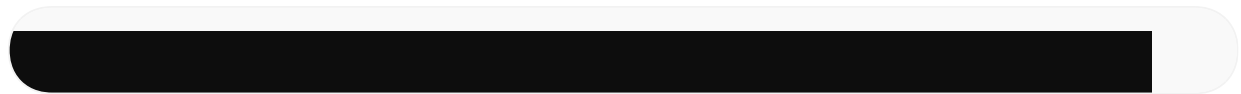
```
io.  "/" .  "join-room"  room  id  
console.  `socket  id  joined  room `
```

Example:



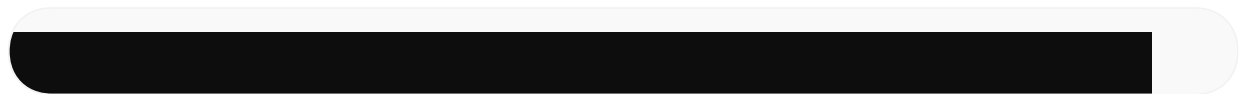
leave-room

Triggered when a socket leaves.



delete-room

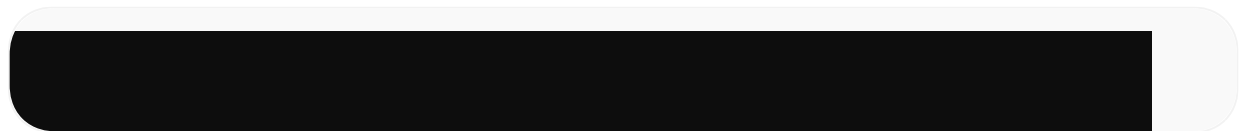
When the last socket leaves.



9 What the Emitter is

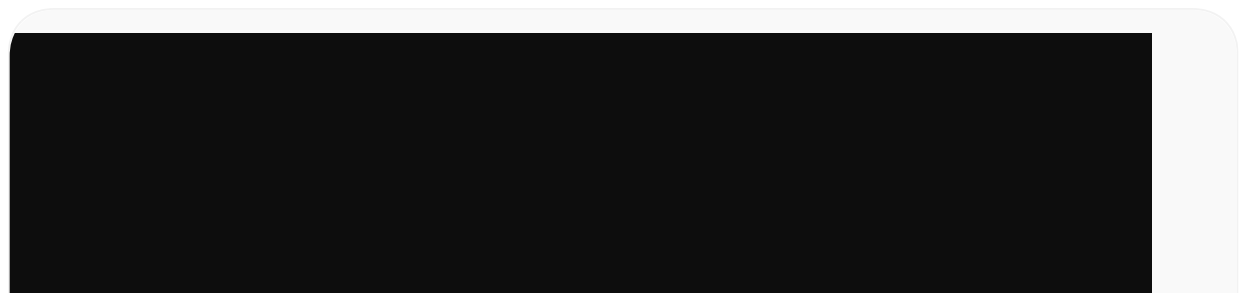
The **Emitter** lets another Node.js service send events to Socket.IO servers.

Example architecture:



M2 can broadcast using the emitter.

Diagram:



10 Example emitter usage

Send to all clients:

```
</> JavaScript
emitter.      "message" "hello"
```

Send to room:

```
</> JavaScript
emitter.  "room1" .    "message" "hello"
```

Send to room except another room:

```
</> JavaScript
emitter.  "room1" .    "room2" .    "message" "hello"
```

1 1 Managing sockets remotely

Emitter can also control sockets.

Join sockets to a room

```
</> JavaScript
emitter.      "room1"
```

All sockets join room1.

Leave rooms

```
</> JavaScript
emitter.      "room1"
```

Disconnect sockets


```
</> JavaScript
```

```
emitter.
```

Disconnect everyone.

Or specific room:

```
</> JavaScript
```

```
emitter.  "room1" .
```

1 2 serverSideEmit()

This sends events **between** Socket.IO servers.

Example:

```
</> JavaScript
```

```
emitter.  "hello" "world"
```

Server receives:

```
</> JavaScript
```

```
io.  "hello"  arg  
console.  arg
```

Output:

This is **server-to-server** communication.

Big Picture

Adapter architecture:

Adapters ensure:

✅ Simple mental model

💡 If you'd like, I can also explain something **very important but rarely explained clearly**:

How Socket.IO internally stores rooms using two Maps (`rooms` and `sids`) and how broadcasting works in $O(n)$ time.

That insight makes the adapter architecture **much easier to understand**.

now move forward to this

Redis adapter

How it works

The Redis adapter relies on the Redis Pub/Sub mechanism.

Every packet that is sent to multiple clients (e.g. `io.to("room1").emit()` or `socket.broadcast.emit()`) is:

sent to all matching clients connected to the current server

published in a Redis channel, and received by the other Socket.IO servers of the cluster

Diagram of how the Redis adapter works

The source code of this adapter can be found [here](#).

Supported features

Feature socket.io version Support

Socket management 4.0.0 ✅ YES (since version

6.1.0)

Inter-server communication 4.1.0  YES (since version 7.0.0)

Broadcast with acknowledgements 4.5.0  YES (since version 7.2.0)

Connection state recovery 4.6.0  NO

Installation

npm install @socket.io/redis-adapter

Compatibility table

Redis Adapter version Socket.IO server version

4.x 1.x

5.x 2.x

6.0.x 3.x

6.1.x 4.x

7.x and above 4.3.1 and above

Usage

tip

For new developments, we recommend using the sharded adapter, which takes advantage of the sharded Pub/Sub feature introduced in Redis 7.0.

With the redis package

caution

The redis package seems to have problems restoring the Redis subscriptions after reconnection:

<https://github.com/redis/node-redis/issues/2155>

<https://github.com/redis/node-redis/issues/1252>

You may want to use the ioredis package instead.

```
import { createClient } from "redis";
import { Server } from "socket.io";
import { createAdapter } from "@socket.io/redis-adapter";
```

```
const pubClient = createClient({ url:
"redis://localhost:6379" });
const subClient = pubClient.duplicate();
```

```
await Promise.all([
  pubClient.connect(),
  subClient.connect()
]);
```

```
const io = new Server({  
  adapter: createAdapter(pubClient, subClient)  
});
```

```
io.listen(3000);
```

With the redis package and a Redis cluster

```
import { createCluster } from "redis";  
import { Server } from "socket.io";  
import { createAdapter } from "@socket.io/redis-  
adapter";
```

```
const pubClient = createCluster({  
  rootNodes: [  
    {  
      url: "redis://localhost:7000",  
    },  
    {  
      url: "redis://localhost:7001",  
    },  
    {  
      url: "redis://localhost:7002",  
    },  
  ],  
});  
const subClient = pubClient.duplicate();
```

```
await Promise.all([  
  pubClient.connect(),  
  subClient.connect()  
]);
```

```
const io = new Server({  
  adapter: createAdapter(pubClient, subClient)  
});
```

```
io.listen(3000);
```

With the ioredis package

```
import { Redis } from "ioredis";  
import { Server } from "socket.io";  
import { createAdapter } from "@socket.io/redis-  
adapter";
```

```
const pubClient = new Redis();
const subClient = pubClient.duplicate();

const io = new Server({
  adapter: createAdapter(pubClient, subClient)
});

io.listen(3000);
```

With the ioredis package and a Redis cluster

```
import { Cluster } from "ioredis";
import { Server } from "socket.io";
import { createAdapter } from "@socket.io/redis-adapter";
```

```
const pubClient = new Cluster([
  {
    host: "localhost",
    port: 7000,
  },
  {
    host: "localhost",
    port: 7001,
  },
  {
    host: "localhost",
    port: 7002,
  },
]);
const subClient = pubClient.duplicate();

const io = new Server({
  adapter: createAdapter(pubClient, subClient)
});

io.listen(3000);
```

With Redis sharded Pub/Sub

Sharded Pub/Sub was introduced in Redis 7.0 in order to help scaling the usage of Pub/Sub in cluster mode.

Reference:

<https://redis.io/docs/interact/pubsub/#sharded-pubsub>

A dedicated adapter can be created with the `createShardedAdapter()` method:

With redis

Minimum requirements:

Redis 7.0

redis@4.6.0

```
import { Server } from "socket.io";  
import { createClient } from "redis";  
import { createShardedAdapter } from "@socket.io/redis-adapter";
```

```
const pubClient = createClient({ host: "localhost", port: 6379 });  
const subClient = pubClient.duplicate();
```

```
await Promise.all([  
  pubClient.connect(),  
  subClient.connect()  
]);
```

```
const io = new Server({  
  adapter: createShardedAdapter(pubClient, subClient)  
});
```

```
io.listen(3000);
```

With ioredis

Minimum requirements:

Redis 7.0

ioredis@5.9.0

Please note that the `shardedSubscribers` option is required to enable sharded Pub/Sub.

```
import { Cluster } from "ioredis";  
import { Server } from "socket.io";  
import { createShardedAdapter } from "@socket.io/redis-adapter";
```

```
const pubClient = new Cluster(  
  [  

```

```

    {
      host: "localhost",
      port: 7000,
    },
    {
      host: "localhost",
      port: 7001,
    },
    {
      host: "localhost",
      port: 7002,
    },
  ],
  {
    shardedSubscribers: true,
  }
);
const subClient = pubClient.duplicate();

const io = new Server({
  adapter: createShardedAdapter(pubClient, subClient)
});

io.listen(3000);

```

Options

Default adapter

Name	Description	Default value
key	The prefix for the Redis Pub/Sub channels.	socket.io
requestsTimeout	After this timeout the adapter will stop waiting from responses to request.	5_000
publishOnSpecificResponseChannel	Whether to publish a response to the channel specific to the requesting node.	false
parser	The parser to use for encoding and decoding messages sent to Redis.	-

tip

Setting the `publishOnSpecificResponseChannel` option to `true` is more efficient since the responses (for example when calling `fetchSockets()` or `serverSideEmit()`) are only sent to the requesting server, and not to all the servers.

However, it currently defaults to `false` for backward-

compatibility.

Sharded adapter

Name	Description	Default value
channelPrefix	The prefix for the Redis Pub/Sub channels.	socket.io
subscriptionMode	The subscription mode impacts the number of Redis Pub/Sub channels used by the adapter.	dynamic

Available values for the subscriptionMode option:

Value	# of Pub/Sub channels	Description
static	2 per namespace	Useful when used with dynamic namespaces.
dynamic (default)	(2 + 1 per public room) per namespace	Useful when some rooms have a low number of clients (so only a few Socket.IO servers are notified).
dynamic-private	(2 + 1 per room) per namespace	Like dynamic but creates separate channels for private rooms as well. Useful when there is lots of 1:1 communication via socket.emit() calls.

Common questions

Is there any data stored in Redis?

No, the Redis adapter uses the Pub/Sub mechanism to forward the packets between the Socket.IO servers, so there are no keys stored in Redis.

Do I still need to enable sticky sessions when using the Redis adapter?

Yes. Failing to do so will result in HTTP 400 responses (you are reaching a server that is not aware of the Socket.IO session).

More information can be found [here](#).

What happens when the Redis server is down?

In case the connection to the Redis server is severed, the packets will only be sent to the clients that are connected to the current server.

Migrating from socket.io-redis

The package was renamed from socket.io-redis to @socket.io/redis-adapter in v7, in order to match the name of the Redis emitter (@socket.io/redis-emitter).

To migrate to the new package, you'll need to make sure to provide your own Redis clients, as the package will no longer create Redis clients on behalf of the user.

Before:

```
const redisAdapter = require("socket.io-redis");

io.adapter(redisAdapter({ host: "localhost", port: 6379 }));
```

After:

```
const { createClient } = require("redis");
const { createAdapter } = require("@socket.io/redis-adapter");

const pubClient = createClient({ url:
"redis://localhost:6379" });
const subClient = pubClient.duplicate();

io.adapter(createAdapter(pubClient, subClient));
```

tip

The communication protocol between the Socket.IO servers has not been updated, so you can have some servers with `socket.io-redis` and some others with `@socket.io/redis-adapter` at the same time.

Latest releases

Version	Release date	Release notes	Diff
8.3.0	March 2024	link	8.2.1...8.3.0
8.2.1	May 2023	link	8.2.0...8.2.1
8.2.0	May 2023	link	8.1.0...8.2.0
8.1.0	February 2023	link	8.0.0...8.1.0
8.0.0	December 2022	link	7.2.0...8.0.0
7.2.0	May 2022	link	7.1.0...7.2.0

[Complete changelog](#)

Emitter

The Redis emitter allows sending packets to the connected clients from another Node.js process:

Diagram of how the Redis emitter works

This emitter is also available in several languages:

Javascript: <https://github.com/socketio/socket.io-redis-emitter>

Java: <https://github.com/sunsus/socket.io-java-emitter>

Python: <https://pypi.org/project/socket.io-emitter/>

PHP: <https://github.com/rase-/socket.io-php-emitter>

Golang: <https://github.com/yosuke-furukawa/socket.io-go-emitter>

Perl: <https://metacpan.org/pod/SocketIO::Emitter>

Rust: <https://github.com/epli2/socketio-rust-emitter>

Installation

```
npm install @socket.io/redis-emitter redis
```

Usage

```
import { Emitter } from "@socket.io/redis-emitter";
import { createClient } from "redis";
```

```
const redisClient = createClient({ url:
"redis://localhost:6379" });
```

```
redisClient.connect().then(() => {
  const emitter = new Emitter(redisClient);
```

```
  setInterval(() => {
    emitter.emit("time", new Date);
  }, 5000);
});
```

Note: with redis@3, calling connect() on the Redis client is not needed:

```
import { Emitter } from "@socket.io/redis-emitter";
import { createClient } from "redis";
```

```
const redisClient = createClient({ url:
"redis://localhost:6379" });
const emitter = new Emitter(redisClient);
```

```
setInterval(() => {
  emitter.emit("time", new Date);
}, 5000);
```

Please refer to the cheatsheet here.

Migrating from socket.io-emitter

The package was renamed from socket.io-emitter to @socket.io/redis-emitter in v4, in order to better reflect the relationship with Redis.

To migrate to the new package, you'll need to make sure to provide your own Redis clients, as the package will no longer create Redis clients on behalf of the user.

Before:

```
const io = require("socket.io-emitter")({ host: "127.0.0.1",
port: 6379 });
```

After:

```
const { Emitter } = require("@socket.io/redis-emitter");
const { createClient } = require("redis");
```

```
const redisClient = createClient();
const io = new Emitter(redisClient);
```

Latest releases

Version	Release date	Release notes	Diff
5.1.0	January 2023	link	5.0.0...5.1.0
5.0.0	September 2022	link	4.1.1...5.0.0
4.1.1	January 2022	link	4.1.0...4.1.1
4.1.0	May 2021	link	4.0.0...4.1.0
4.0.0	March 2021	link	3.2.0...4.0.0

This documentation explains **how the Redis adapter works internally with Socket.IO and Redis when you scale to multiple servers.**

I'll break it down step-by-step so you understand the **real architecture and flow.**

1 What the Redis Adapter Does

The **Redis adapter** connects multiple Socket.IO servers together.

Normally:

Without Redis, a broadcast only reaches clients **on the same server**.

Example:

```
</> JavaScript
```

```
io. "room1" . "message" "hello"
```

Result without Redis:

✖
✖

2 How the Redis Adapter Works

The adapter uses **Redis Pub/Sub**.

Redis Pub/Sub means:

Architecture:

Step-by-step flow

User sends a message.

Server1 broadcasts:

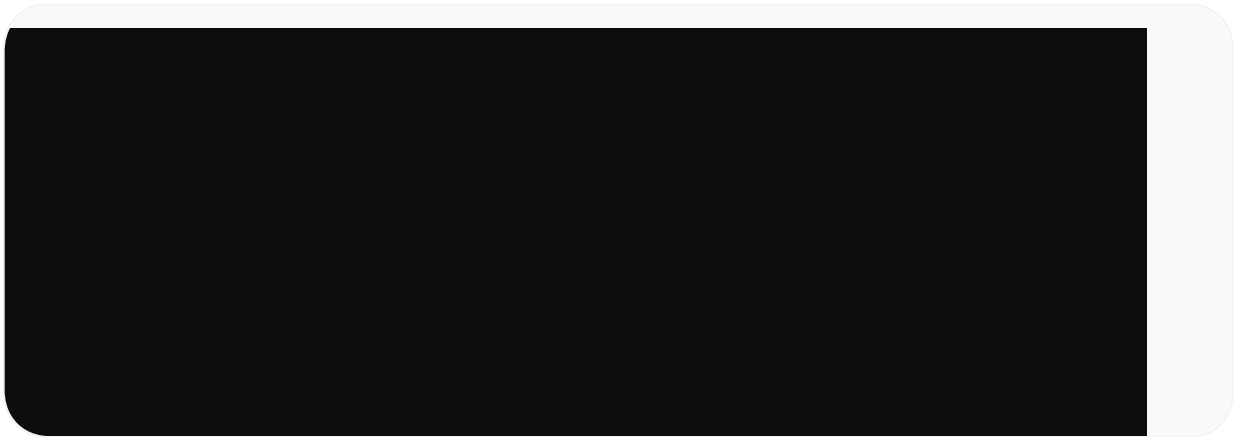
```
</> JavaScript
```

```
io. "room1" . "message" "hello"
```

What happens internally:

- 1 Server1 sends event to its local clients
- 2 Server1 publishes event to Redis
- 3 Redis forwards event to other servers
- 4 Other servers broadcast to their clients

Diagram:

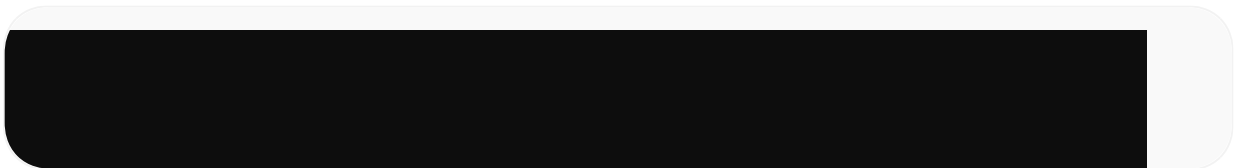


Now **all users receive the event**, no matter which server they are connected to.

3 Why Redis Pub/Sub Is Used

Redis Pub/Sub is extremely fast.

Typical flow:



No data is stored.

The message simply **passes through Redis**.

4 Important Note from Docs

The docs say:

No data is stored in Redis.

Meaning Redis is **not** used as a database here.

It only forwards packets.

5 Installing the Redis Adapter

Install package:

```
</> Bash
```

```
npm
```

6 Basic Setup

Example:

```
</> JavaScript
```

```
import  createClient  from "redis"
import  Server        from "socket.io"
import  createAdapter  from "@socket.io/redis-adapter"

const pubClient = createClient      "redis://localhost:6379"
const subClient = pubClient.

await pubClient.
await subClient.

const io = new Server
          createAdapter pubClient  subClient

io.      3000
```

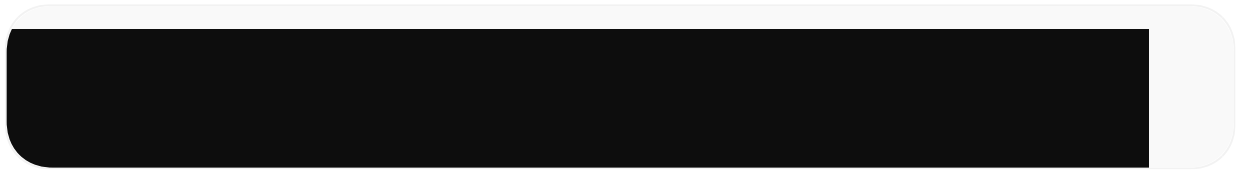
Why two Redis clients?

You need two connections:

Client	Purpose
pubClient	publishes events

Client	Purpose
subClient	subscribes to events

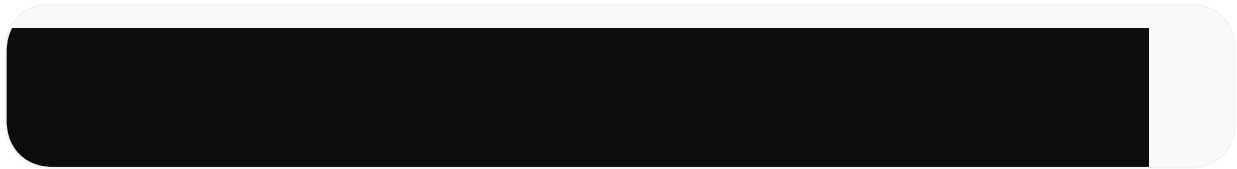
Architecture:



7 Redis Cluster Support

For very large systems, Redis itself can scale.

Example cluster:



This distributes Pub/Sub load.

Example configuration:

JavaScript

```
const pubClient = createCluster  
    ["redis://localhost:7000"  
    "redis://localhost:7001"  
    "redis://localhost:7002"]
```

8 Redis Sharded Pub/Sub (New Feature)

Redis 7 introduced **Sharded Pub/Sub**.

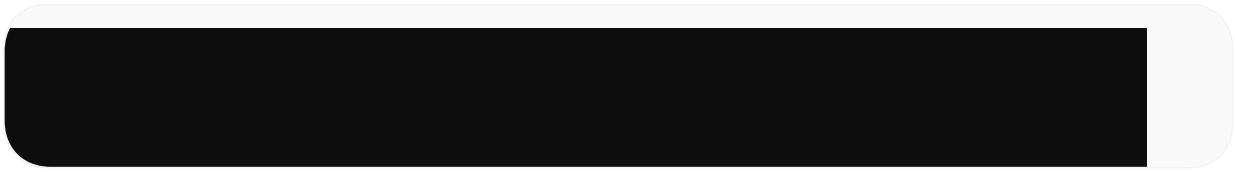
Problem before:



This can overload Redis.

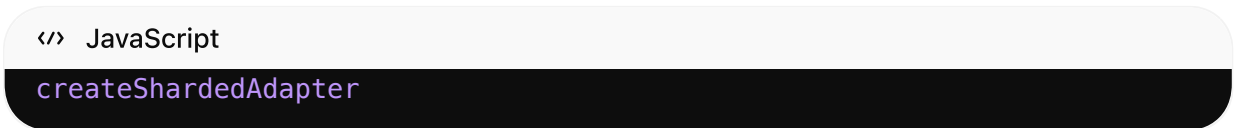
Sharding distributes channels.

Example:



This spreads load across Redis cluster nodes.

Socket.IO supports this with:



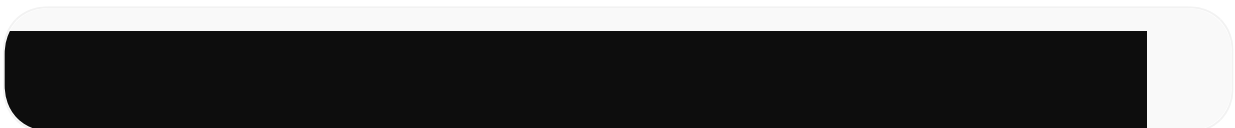
9 Adapter Options

Some configuration options exist.

key

Prefix for Redis channels.

Example:



requestsTimeout

How long servers wait for responses.

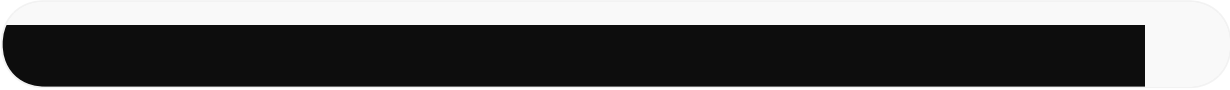
Default:



publishOnSpecificResponseChannel

Optimization.

If enabled:



Instead of sending responses to every server.

10 Subscription Modes (Sharded Adapter)

Redis sharding can use different subscription strategies.

static



Simple but less optimized.

dynamic (default)



Servers only receive events for rooms they care about.

Better performance.

dynamic-private

Creates channels even for **private socket messages**.

Useful when there is a lot of **1-to-1 communication**.

11 Sticky Sessions Requirement

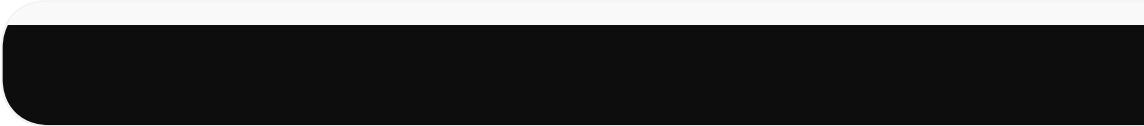
Even with Redis adapter, you **still need sticky sessions**.

Why?

Because Socket.IO sometimes uses **HTTP polling**.

Polling requests must hit the **same server**.

Example without sticky sessions:

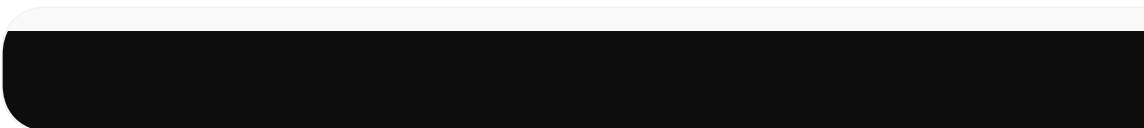


Server2 does not know the session → error.

Result:



Sticky sessions ensure:



1 2 What Happens if Redis Goes Down?

The docs explain this.

If Redis is offline:



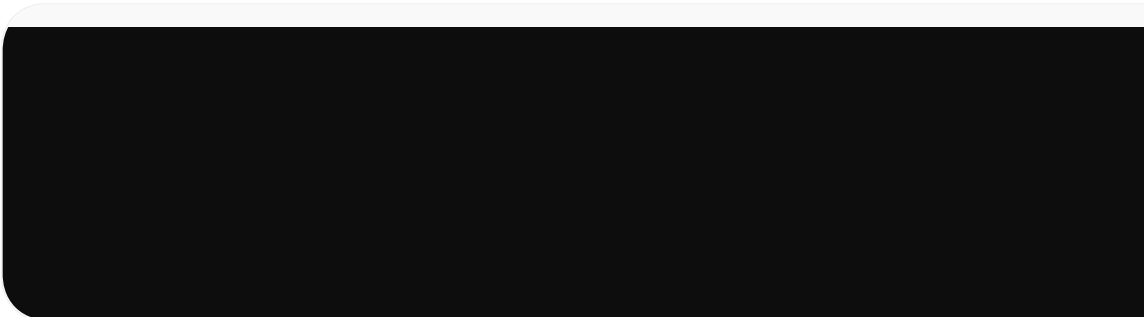
Servers can no longer communicate.

Your app still works but **cross-server broadcasting stops**.

1 3 Redis Emitter

The **Redis emitter** allows **another service** to send events to Socket.IO.

Example architecture:



Example code:

</> JavaScript

```
import Emitter from "@socket.io/redis-emitter"
import createClient from "redis"

const redisClient = createClient
await redisClient.

const emitter = new Emitter redisClient

emitter. "time" new Date
```

Now all clients receive:

</> JavaScript

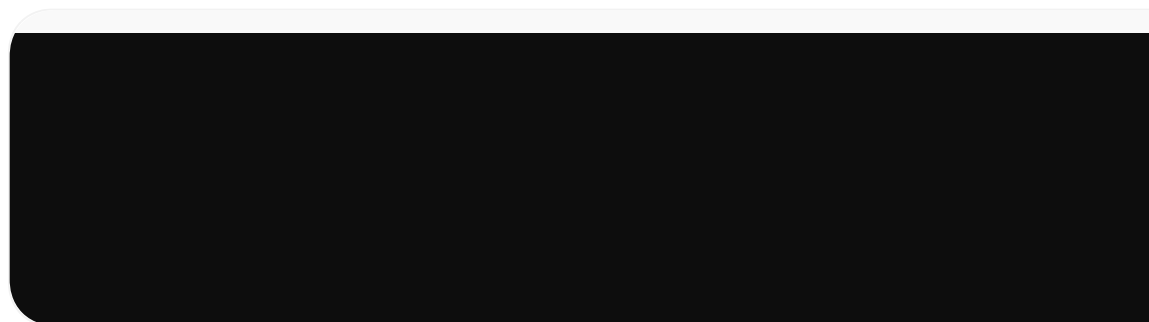
```
socket. "time" data
console. data
```

1 4 Example Real-World Architecture

Large realtime system:



Flow:

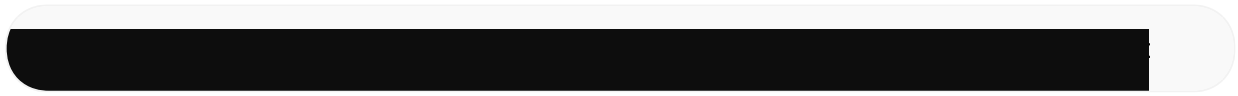


Key Takeaways

Redis adapter solves one major problem:



Core idea:



✅ One-sentence summary

The Redis adapter turns multiple Socket.IO servers into one logical realtime system.

If you'd like, I can also explain the internal algorithm Socket.IO uses to broadcast to rooms (rooms map and sids map).

Understanding that makes the Redis adapter implementation much easier to understand.